Lab 1: Fundamentals of C++

Syntax Refresh, Pointer Mechanics, Array Layouts & STL Bridge

Yohan Fougerolle

Autumn 2026

Preamble & Pedagogical Intent

This first lab helps you regain solid footing in C++. We review fundamental building blocks: function declarations, parameter passing mechanisms (by value, reference, or pointer), returning single/multiple values, control flow syntax, and low-level memory mechanics.

Additionally, mastering low-level memory layout and raw pointer manipulations is mandatory for writing high-performance geometry processing algorithms (such as **Half-Edge mesh structures**). This lab intentionally explores **legacy C-style practices** (`new`, `delete`, raw pointers, `double**`) to expose you to memory layout, pointer arithmetic, and potential pitfalls before introducing modern STL abstractions.

A Note on Pace and Methodology

Do not be intimidated by the length of this lab document! While comprehensive, every section addresses fundamental concepts designed to build your confidence step-by-step. We will work through these exercises across multiple lab sessions, so **take your time** to thoroughly digest each topic.
If you encounter any doubt or edge case, **do not hesitate to ask for extra explanations, code samples, or visual memory diagrams**.

Contents

1 C++ Fundamentals: Syntax, Functions, and Parameters **2**

2 Control Structures and Syntax **3**

3 Variables, References, and Pointer Mechanics **4**

4 Arrays in C++: Static, Dynamic, and Multidimensional **7**

5 Preparation for Complex Structures: Linked Pointers **8**

6 The STL Redemption: Legacy C vs Modern C++ **9**

1 C++ Fundamentals: Syntax, Functions, and Parameters

Concept Review

In C++, functions require explicit return types and parameter declarations. Parameters can be passed:

- **By value** (`double x`): a local copy of the argument is created inside the function stack frame. The original caller variable remains strictly untouched, even if the parameter is reassigned locally.

```cpp
double computeArea(double radius) {
    constexpr double pi = 3.14159;
    double area = pi * radius * radius;

    // Suppose we want to update radius once area is computed...
    radius = radius * 2.0;

    // 'radius' is now twice the value passed to the function,
    // but the initial variable in the caller remains unchanged!
    return area;
}
```

- **By pointer** (`int* p`): passes the memory address of the caller's variable. Requires explicit dereferencing (`*p`) to mutate the underlying data.

```cpp
void updateValue(int* ptr) {
    if (ptr != nullptr) {
        *ptr = 42; // Mutates the original caller variable
    }
}
```

- **By reference** (`int& ref`): creates an immutable alias to the caller's variable. Allows direct modification with clean syntax (no dereferencing required).

```cpp
void increment(int& val) {
    val += 1; // Mutates the original caller variable directly
}
```

- **By const reference** (`const std::string& ref`): creates a read-only alias. Avoids the overhead of copying large objects while preventing modifications.

```cpp
void displayMessage(const std::string& msg) {
    // msg += "!"; // Compiler error: read-only reference
    std::cout << msg << std::endl;
}
```

Returning Multiple Values

To return multiple values from a function in C++, you can:

1. Use output parameters passed by reference (`T& out1, U& out2`).
2. Use output parameters passed by pointer (`T* out1, U* out2`).
3. Use standard structured types such as `std::pair<T, U>` or `std::tuple<...>` (C++11/17).
4. Define a dedicated `struct` containing the return fields.

Exercise 1. Parameter Passing Mechanics Write four separate functions that increment an integer by a given offset:

1. `int incByValue(int val, int offset)` — Returns the new incremented value.
2. `void incByPointer(int* val, int offset)` — Modifies the value directly via a raw pointer.
3. `void incByReference(int& val, int offset)` — Modifies the value directly via a reference.
4. Call all three functions in your `main()` and observe which caller variables are modified.

Exercise 2. Returning Multiple Values (Min and Max) Implement three different versions of a function that computes both the minimum and maximum of two integers:

1. `void minMaxRef(int a, int b, int& outMin, int& outMax)`
2. `void minMaxPtr(int a, int b, int* outMin, int* outMax)`
3. `std::pair<int, int> minMaxPair(int a, int b)`

Test all three signatures in `main()` and print the results.

2 Control Structures and Syntax

Syntax Reference

```cpp
// Conditional Branching
if (condition) { /* ... */ } else if (otherCondition) { /* ... */ } else {
    /* ... */ }

switch (integerOrEnum) {
    case 1:  /* ... */ break;
    default: /* ... */ break;
}

// Loops
for (int i = 0; i < N; ++i) { /* ... */ }
while (condition) { /* ... */ }
do { /* ... */ } while (condition);
```

Exercise 3. Conditional Structures (`if/else` & `switch`) Write a function `void evaluateGrade(int score)` that:

1. Uses an `if / else if / else` ladder to print "Fail" ($score < 10$), "Pass" ($10 \leq score < 14$), or "Excellent" ($score \geq 14$).
2. Defines an `enum class State { Idle, Processing, Success, Error }` and writes a `switch` statement handling every state with a dedicated message.

Exercise 4. Loop Syntax (`for`, `while`, `do-while`) Write three separate loop implementations to calculate the factorial of a given integer N:

1. A `for` loop iteration.
2. A `while` loop iteration.
3. A `do-while` loop iteration.

3 Variables, References, and Pointer Mechanics

Concept Review: What is a Pointer?

At its core, **a pointer is simply a standard variable whose stored value happens to be a memory address**. Like any other variable, a pointer occupies a specific slot in memory and has its own memory address.

Three Ways to Obtain an Address in a Pointer

1. **Pointing to an existing static/stack variable:** Using the address-of operator (`&`), we retrieve the memory location of a variable on the stack.

```
int a = 42;
int* pa = &a; // pa stores the memory address of 'a'
```

2. **Pointing to a newly allocated heap memory block (Single Object):** Using `new`, the system reserves a single memory block on the heap and returns its starting address.

```
int* b = new int; // b stores the address of a newly allocated integer
*b = 100;         // Writes 100 directly into that heap space
delete b;         // Mandatory cleanup!
```

3. **Pointing to a dynamic array (Multiple Contiguous Objects):** Using `new[]`, the system allocates a contiguous sequence of elements on the heap. The returned pointer points directly to the **first element** (`&c[0]`).

```
int* c = new int[10]; // c points to the 1st element of a 10-int array
c[0] = 5;             // Access using array indexing
*(c + 1) = 12;        // Equivalent access using pointer arithmetic (c[1])
delete[] c;           // Mandatory cleanup with delete[]!
```

Memory Architecture: Stack vs. Heap

When a C++ program executes, the operating system assigns it a virtual memory space divided into several distinct segments. Understanding where variables reside is fundamental to managing memory correctly and preventing catastrophic runtime failures.

The Dual Memory Model

- **The Stack (Stack Frame Allocation):**
  - **Purpose:** Stores local variables, function arguments, and return addresses.
  - **Management:** Managed automatically by the CPU via the Stack Pointer (`SP`). Memory is allocated when entering a scope and instantly released upon exit.
  - **Characteristics:** Blazing fast access (L1/L2 cache-friendly), but strictly limited in size (typically 1–8 MB). Exceeding this limit triggers a **Stack Overflow**.
- **The Heap (Dynamic Allocation Pool):**
  - **Purpose:** Stores large datasets, dynamically-sized structures, and long-lived objects whose lifetime spans beyond local scopes.
  - **Management:** Managed manually by the programmer using `new` / `delete` (or automatically via modern RAII - *Resource Acquisition Is Initialization* containers).
  - **Characteristics:** Vast capacity (bounded only by physical RAM and virtual memory), but incurs OS allocation overhead and potential fragmentation. Failing to release memory leads to **Memory Leaks**.

```cpp
void memoryDemo() {
    // 1. Stack Allocation (Automatic lifetime)
    int stackVal = 42;              // 4 bytes stored on the stack
    int stackArray[100];            // ~400 bytes allocated on the stack frame

    // 2. Heap Allocation (Manual lifetime)
    // 'heapPtr' (the local variable holding the address) lives on the Stack!
    // The actual 1,000,000 integers reside on the Heap.
    int* heapPtr = new int[1000000];

    // Accessing heap data via stack pointer dereferencing:
    heapPtr[0] = 123;

    // Manual cleanup required!
    delete[] heapPtr; // If forgotten, those 4 MB remain leaked on the Heap!
} // 'stackVal', 'stackArray', and 'heapPtr' are automatically destroyed here!
```

Exercise 5. Stack vs. Heap Behavior & Lifetime Investigation

1. **Stack Lifetime Trap:** Write a function `int* createDanglingStackInt()` that declares a local variable `int x = 100;` on the stack and returns its address (`&x`). In `main()`, call the function, print the dereferenced return pointer, and explain why this code causes undefined behavior.
2. **Heap Lifetime Persistence:** Write a function `int* createValidHeapInt()` that allocates an integer initialized to `100` on the heap using `new` and returns its pointer. Verify in `main()` that the memory remains valid across function calls until explicitly released with `delete`.
3. **Stack Overflow Experimentation (Optional):** Write a recursive function `void causeStackOverflow(int depth)` that allocates a 100 KB local stack array on each call without a termination base case. Observe the segmentation fault / crash and determine the approximate recursion depth supported by your system's stack.

Beyond Data: Pointers to Functions

Since executable instructions reside in memory, a pointer can store the **starting address of a function**.

```
int add(int x, int y) { return x + y; }

// Declaring a pointer to a function:
int (*funcPtr)(int, int) = &add;

// Executing via pointer:
int result = funcPtr(3, 4); // Evaluates to 7
```

What is a Reference?

A **reference** (`T&`) is an alias for an existing variable. It cannot be `nullptr`, must be initialized upon creation, cannot be reassigned to another target variable, and avoids dereferencing syntax (`*`).

Exercise 6. Progressive Swap: From Values to Pointer Aliases The goal of this exercise is to demystify pointer manipulation syntax step-by-step.

1. **Naive Swap (By Value):** Write `void swapValuesFail(int a, int b)` and explain why caller variables remain unchanged.
2. **Standard Swap (By Reference vs By Pointer):** Implement working versions:
   - `void swapRef(int& a, int& b)`
   - `void swapPtr(int* a, int* b)`
3. **Advanced Swap (Swapping Memory Addresses):** Given two heap pointers `int* p1 = new int(10);` and `int* p2 = new int(20);`:

```
void swapPointers(int** a, int** b);
void swapPointersRef(int*& a, int*& b);
```

   Swap the pointers themselves so that `p1` points to `20` and `p2` points to `10`. Compare both signatures in `main()`.

Exercise 7. Function Pointers as Callbacks

1. Write two basic mathematical functions: `int square(int x)` and `int cube(int x)`.
2. Write a higher-order function `void applyOperation(int* array, int size, int (*op)(int))` that iterates over an array and applies `op` to each element in place.
3. Test your function in `main()` first by passing `square`, then `cube`.

Exercise 8. Const-Correctness Challenge Demonstrate the difference between:

- `const int* p` (pointer to constant integer)
- `int* const p` (constant pointer to integer)
- `const int* const p` (constant pointer to constant integer)

Write a short snippet trying to modify both the address and the underlying value for each case, and observe compiler error messages.

Pointer Arithmetic and Iteration Mechanics

In C++, adding an integer k to a pointer does not simply increment its raw byte address by k. Instead, the address is scaled by the size of the pointed data type:

$$\text{address}(p + k) = \text{address}(p) + k \times \text{sizeof}(T)$$

This mechanism allows traversing contiguous memory blocks (like static or dynamic arrays) without using array indexing brackets (`[]`).

Key Operations in Pointer Arithmetic

- **Increment / Decrement (`p++`, `--p`):** Advances or rewinds the pointer to the next/previous element in memory.
- **Offset addition/subtraction (`p + k`):** Directly jumps k elements forward or backward.
- **Pointer Difference (`p2 - p1`):** Yields the number of elements (of type `T`) separating two pointers within the same contiguous array (type `std::ptrdiff_t`).

```cpp
int numbers[5] = {10, 20, 30, 40, 50};
int* ptr = numbers; // Points to numbers[0]

// Traversing using pointer incrementation
for (int i = 0; i < 5; ++i) {
    std::cout << *ptr << " "; // Dereference current element
    ptr++;                    // Move pointer to the next int (+4 bytes)
}

// Pointer distance
int* start = &numbers[0];
int* end   = &numbers[4];
std::ptrdiff_t count = end - start; // Evaluates to 4 elements
```

Exercise 9. Array Traversal via Pointer Arithmetic In this exercise, you are strictly forbidden from using bracket indexing (`arr[i]`).

1. Write a function `int sumArrayPtr(const int* begin, const int* end)` that calculates the sum of all integers between pointers `begin` (inclusive) and `end` (exclusive) using a pointer-incrementing `while` loop (`while (begin != end)`).
2. Write a function `void reverseArrayPtr(int* begin, int* end)` that reverses an array in-place using two pointers (`front` starting at the beginning, `back` starting at the end) moving toward each other until they meet.
3. In `main()`, test both functions on a statically allocated array of 6 integers. Note how this pointer-range syntax (`[begin, end)`) directly mirrors modern **C++ STL iterators**!

4 Arrays in C++: Static, Dynamic, and Multidimensional

Concept Review

An array is a contiguous block of memory holding elements of the same type. Understanding how arrays degrade into raw pointers (*pointer decay*) when passed to functions is essential for mastering C++ memory mechanics.

Static vs Dynamic Arrays

- **Static Arrays (Stack Allocation):** Fixed compile-time size.

```
int staticArr[5] = {10, 20, 30, 40, 50};
```

- **Dynamic Arrays (Heap Allocation):** Runtime size determination.

```
int size = 10;
int* dynArr = new int[size];
delete[] dynArr; // Mandatory cleanup!
```

Passing Arrays To and From Functions

When passing a raw array to a function, it decays into a pointer to its first element (`int*`). Array size information is lost and must be passed as an explicit parameter.

Dangling Pointer Hazard

Never return a pointer to a local stack array! The stack frame is destroyed upon function exit. To return an array, allocate on the heap (`new[]`) or pass a pre-allocated output buffer.

Multidimensional Arrays

1. **Array of Pointers (`T**`):** Non-contiguous row allocations on the heap.
2. **Flat 1D Buffer (`T*`):** Single contiguous block with offset arithmetic index $= i \times \text{cols} + j$.

Exercise 10. Array Decay and Function Transfers

1. Write a function `void fillRandom(int* arr, int size, int minVal, int maxVal)` populating an array with random values.
2. Write `int* createFactorialArray(int n)` allocating a heap array filled with factorials $(0!, 1!, \ldots, (n-1)!)$.
3. Call it in `main()`, print the values, and properly `delete[]` the memory.

Exercise 11. Flattened 2D Matrix Processing

1. Write `void transposeFlatMatrix(const double* src, double* dst, int rows, int cols)` for 1D heap allocations.
2. Write `void printFlatMatrix(const double* mat, int rows, int cols)`.
3. Test your functions in `main()` with a 3×4 matrix.

5 Preparation for Complex Structures: Linked Pointers

Concept Review

Advanced topological data structures (like Half-Edge meshes) rely heavily on structs referencing each other through raw pointers.

Exercise 12. Pointers in Linked Topological Structures Consider a simplified topological node representing a directed graph vertex:

```
struct Node {
    int id;
    Node* next;
    Node* prev;
};
```

1. Write a function `void insertAfter(Node* current, Node* newNode)` updating both `next` and `prev` pointers.
2. Write a function `void removeNode(Node* target)` unlinking a node and releasing its memory.
3. Trace manually on paper the pointer reassignments to ensure no pointer becomes dangling or unreferenced.

From Linear Links to Hierarchical Trees

Topological representations (such as Half-Edge meshes or CSG trees) rely on pointers pointing to multiple neighbor structures simultaneously. Managing parent, child, or sibling pointers requires a strict protocol for insertion, traversal, and memory cleanup.

```
struct TreeNode {
    int value;
    TreeNode* left;
    TreeNode* right;

    // Constructor for cleaner allocation
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {}
};
```

Exercise 13. Binary Search Tree (BST) Mechanics

1. **Recursive Insertion:** Write a function `TreeNode* insertBST(TreeNode* root, int val)` that inserts a new value into a Binary Search Tree according to standard BST rules ($val < root \to left$, $val > root \to right$). Memory must be dynamically allocated on the heap for new nodes.
2. **In-Order Traversal:** Write a function `void printInOrder(const TreeNode* root)` that prints tree values in ascending order.
3. **Recursive Tree Deallocation (Memory Cleanup Challenge):** Write a function `void freeTree(TreeNode* root)` that releases every node in the tree using a post-order traversal ($left \to right \to root$). Explain why deleting the root before its children results in an unreachable memory leak.
4. **Testing in `main()`:** Construct a tree with values `{15, 6, 23, 4, 7, 71}`, print its ordered content, and verify that your deallocation function cleanly releases all allocated heap nodes without triggering undefined behavior.

6 The STL Redemption: Legacy C vs Modern C++

While a comprehensive section of the course will be fully dedicated to the Standard Template Library (STL) later on, this brief overview is provided so you can immediately adopt modern C++ abstractions and start writing clean, safe code.

Concept Review: The Pain of Legacy C vs STL Simplicity

Manual heap management requires extreme care to avoid buffer overflows, memory leaks, and dangling pointers. Modern C++ relies on RAII containers like `std::vector` to manage memory automatically.

Legacy C vs STL Equivalent Syntax

- **Legacy Resizable Array:** Requires manual allocation, element-by-element copy, and explicit deallocation.
- **Modern STL Vector:** Memory allocation, growth strategy, and deallocation are handled automatically.

Exercise 14. Comparative Challenge: Legacy C vs Modern STL

For each sub-problem below, write two distinct implementations:

- **Version A (Legacy C Style):** Use raw pointers, explicit `new[]` / `delete[]`, and manual loop indexing.
- **Version B (Modern C++ STL Style):** Use `std::vector`, standard algorithms, and range-based loops.

1. **1D Array Copy & Growth:** Write a function that takes an array, creates a duplicate with twice its size, copies the original elements over, initializes the remaining elements to zero, and releases the old memory.
2. **Dynamic 2D Matrix Resizing:** Write code allocating an $N \times M$ grid. In Version A, construct an array of pointers (`double**`). In Version B, use a nested vector (`std::vector<std::vector<double>>`). Observe the difference in initialization, access, and cleanup syntax!
3. **Dynamic 3D Volume Grid ($N \times M \times P$):** Write code allocating a 3D grid. Compare allocating `float***` vs a flat `std::vector<float>` using index mapping $i \cdot (M \cdot P) + j \cdot P + k$.
4. **Multiple Return Values Refactoring:** Rewrite a function returning minimum, maximum, and average values. Compare output pointers (`double* outMin, double* outMax, double* outAvg`) against returning a `std::tuple` unpacked with C++17 structured bindings:

```
auto [minVal, maxVal, avgVal] = computeStats(dataVector);
```